Anexo de Trazabilidad de Inteligencia Artificial (IA)

Trabajo Práctico 2 — Seminario de Actualización 2026
Materia: Seminario de Actualización | **Docente:** Jorge Insfran
Unidad 5: Introducción a Data Analytics e Ingeniería de Datos
Equipo: 6 | **Integrantes:** Agustin Bugnoni, Lorenzo Zárate
Variante: 6 (Precios, recupero y rentabilidad del catálogo)

---

1. Metodología de Trabajo: El Paradigma de IA Agente en Google Antigravity

A diferencia de un uso convencional de Inteligencia Artificial basado en consultas aisladas a un chatbot web (donde se solicita código fragmentado mediante copiar y pegar), el desarrollo de este Trabajo Práctico se llevó a cabo utilizando un **Agente de Inteligencia Artificial Autónomo integrado directamente en el entorno de desarrollo (Google Antigravity IDE)**.

Características del Flujo Agéntico

- **Acceso Contextual al Repositorio:** El agente no operó "a ciegas", sino que tuvo capacidad de inspeccionar el árbol del proyecto, leer documentación oficial (enunciado del TP2 y apunte de la Unidad 5 en formato PDF), analizar esquemas SQL y examinar archivos de datos.
- **Supervisión Continua (*Human-in-the-Loop*):** Ningún cambio de código, script o refactorización fue ejecutado de manera desatendida. Asumimos un rol activo de **Dirección y Supervisión Técnica**, estableciendo restricciones de diseño, revisando minuciosamente cada modificación antes de aplicarla al repositorio y sometiendo cada propuesta a un proceso de repregunta e iteración crítica.
- **Ciclo de Desarrollo en Cuatro Fases:**
    1. *Síntesis y Alineación Teórica:* Solicitamos al agente analizar el apunte de la Unidad 5 (`Seminario_2026_Unidad05.pdf`) para extraer y estructurar los conceptos rectores (las 6 dimensiones de calidad de datos, el criterio de Tukey para outliers, el diseño EDA y los patrones de pipelines idempotentes).
    2. *Planificación Estructurada:* Previo a la escritura de código, elaboramos y acordamos un plan integral de implementación (`plan_implementacion_1.md`) que desglosó el proyecto en hitos cronológicos y fijó las salvaguardas técnicas para evitar condiciones de desaprobación.
    3. *Ejecución Punto a Punto con Repreguntas:* Abordamos las Actividades 0 a 5 de manera secuencial, exigiendo justificación técnica a cada decisión (elección de controladores, reparto de cómputo SQL vs. Python, escalas de visualización).
    4. *Auditoría Defensiva y Testing:* Verificamos la consistencia de resultados ejecutando pruebas de idempotencia, reconciliación numérica entre motor y cliente, y validación contra casos de borde.

---

2. Bitácora de Prompts, Repreguntas y Decisiones Supervisadas

A continuación se documenta la secuencia real de interacciones mantenidas con el agente a lo largo de las distintas etapas del proyecto, reflejando cómo nuestra supervisión corrigió desvíos tempranos y direccionó la solución hacia los requerimientos de la cátedra.

Hito 1: Marco Teórico y Síntesis de la Unidad 5

- **Interacción Inicial:**
  Prompt enviado: "Analizá el PDF de la Unidad 5 (`Seminario_2026_Unidad05.pdf`) junto con la consigna del TP2. Necesito un resumen ejecutivo de los conceptos teóricos obligatorios que debemos cumplir en el código y en el informe, especialmente en lo relativo a calidad de datos y arquitectura de pipelines."
  Respuesta del Agente: El agente elaboró un resumen conceptual, pero en una primera instancia incluyó un checklist genérico de analítica web con métricas que no correspondían a la bibliografía de la materia.
- **Intervención y Repregunta:**
  Nuestra corrección: "¿De dónde sacaste ese checklist? La Unidad 5 hace referencia estricta a lo dictado en clase y al apunte oficial. Si algo no está en el material, preguntame; no inventes conceptos externos. Releé el PDF de la Unidad 5 y estructurá el análisis según las 6 dimensiones de calidad formales y el checklist de 8 preguntas."
- **Resultado Consolidado:**
  El agente reorientó su análisis, extrayendo con exactitud las 6 dimensiones formales (Completitud, Unicidad, Validez, Consistencia, Exactitud y Oportunidad) y el marco de detección de anomalías de Tukey (1977), volcando este marco en `Diagnostico_actividad_2.md` y en las celdas de perfilado del notebook `notebooks/analisis.ipynb`.

Hito 2: Plan de Implementación y Arquitectura de Portabilidad (Actividad 0)

- **Interacción Inicial:**
  Prompt enviado: "Generá un plan de implementación detallado (`plan_implementacion_1.md`) para abordar el trabajo práctico. No modifiques código todavía; primero quiero revisar la viabilidad, el cronograma y las decisiones técnicas para blindar la aprobación."
  Propuesta del Agente: El plan inicial propuso consultas T-SQL directas utilizando nombres canónicos de Sakila en inglés (`film`, `inventory`, `rental`, `payment`).
- **Intervención y Repregunta:**
  Observación y objeción del equipo: En el TP1 nuestra base de datos migró sus tablas a nombres en español (`pelicula`, `inventario`, `alquiler`, `pago`). Si se hardcodean nombres en inglés o en español, se viola la Sección 4 (Actividad 0) y Sección 6 (Defensa), que exige que el proyecto corra contra la base de cualquier otro equipo modificando **únicamente** `esquema.py`.
  Nuestra repregunta: "La consigna dice que las tablas se relacionan con sus nombres canónicos, pero en nuestra base están traducidas. Si cambiamos `esquema.py` por el de otro equipo, ¿tu propuesta fallaría? Rediseñá el módulo para que desacople totalmente las queries mediante un constructor dinámico."
- **Decisión Técnica Adoptada:**
  Se implementó en `esquema.py` la función `generar_query_conjunto_trabajo()`, la cual interpola los nombres mapeados desde el diccionario hacia la consulta SQL manteniendo los parámetros de filtrado protegidos mediante `params` en `pd.read_sql`. Esto garantiza portabilidad total y previene inyección SQL.

Hito 3: Reparto de Cómputo y Preservación de Casos de Borde (Actividad 1)

- **Interacción Inicial:**
  Prompt enviado: "Escribí la consulta del conjunto de trabajo para la Variante 6 uniendo películas, inventario, alquileres y pagos, calculando la recaudación total y los alquileres por película. Mostrame cómo dividimos el trabajo entre SQL Server y Pandas."
  Propuesta del Agente: El agente propuso una consulta con `INNER JOIN` entre las cuatro tablas y sugirió calcular los ratios de recupero dentro de SQL Server mediante divisiones en T-SQL.
- **Intervención y Repregunta:**
  Detección de falla: Al auditar el resultado del query propuesto contra la reconciliación de la Actividad 0 (1.000 películas en catálogo), detectamos que la consulta arrojaba solo **958 filas**.
  Le señalamos al agente: "En Sakila existen 42 títulos que no tienen copias registradas en inventario. Si usás `INNER JOIN`, esas 42 películas se descartan silenciosamente. Eso viola el requerimiento de catálogo completo y oculta el costo de oportunidad del negocio. Corregí la consulta a `LEFT JOIN` con `ISNULL(..., 0)`."
  Alineación de arquitectura: Asimismo, corregimos el reparto de trabajo: las agregaciones pesadas y uniones se mantuvieron en el motor (`SUM`, `COUNT DISTINCT`), pero el cálculo de métricas financieras derivadas y el manejo de casos de borde se derivaron a Python para permitir enriquecimiento multi-fuente.

Hito 4: Perfilado de Datos y Tratamiento de Outliers (Actividad 2)

- **Interacción Inicial:**
  Prompt enviado: "Ejecutemos el diagnóstico de calidad de datos sobre el DataFrame extraído. Aplicá el método de Tukey (IQR) sobre los ingresos acumulados y decime qué hacemos con los valores detectados."
  Propuesta del Agente: El agente identificó 11 títulos con ingresos superiores a $181.80 (IQR superior) y 42 títulos con recaudación $0.00. Sugirió filtrar o eliminar ambos grupos afirmando: *"Las películas con ingreso cero son registros incompletos que ensucian la distribución, y los outliers superiores distorsionan la media y sesgan las regresiones"*.
- **Intervención y Criterio Aplicado:**
  Rechazo del descarte automático: Frenamos la sugerencia de la IA aplicando el criterio de dominio de la Unidad 5:
  1. Las películas con $0.00 no son errores de carga: son títulos legítimos del catálogo que no poseían copias en inventario (`inventory_count = 0`). Descartarlas implicaría subestimar el catálogo.
  2. Los 11 outliers superiores (como *TELEGRAPH VOYAGE*, $231.73) representan los *blockbusters* de mayor rotación comercial. En analítica de negocios, eliminar los mayores éxitos de venta constituye un sesgo metodológico grave. *Instrucción dada al agente:* Mantuvimos el 100% de las 1.000 filas, asignando a los títulos sin inventario una categoría explícita (`"Sin inventario"`) y justificando la conservación de los blockbusters en el informe.

Hito 5: Visualización Narrativa y Elección de Escalas (Actividad 3)

- **Interacción Inicial:**
  Prompt enviado: "Vamos a construir el gráfico obligatorio de la Variante 6: dispersión entre precio de alquiler y cantidad de alquileres, con tamaño de burbuja según recaudación."
  Desvío del Agente: El agente generó un scatter plot estándar en Matplotlib donde los puntos aparecían superpuestos verticalmente en tres columnas idénticas, volviendo la visualización ilegible.

- **Supervisión y Repregunta Teórica:**
  Nuestra repregunta: "Quiero que sigamos hablando sobre el Gráfico 1. ¿Cómo hiciste la elección de los ejes y por qué los puntos están colapsados? Respondeme sin tocar código todavía."
  Ajuste metodológico: Tras la discusión técnica, constatamos que debido a que `rental_rate` solo asume 3 valores discretos ($0.99, $2.99, $4.99), era indispensable aplicar **jitter horizontal aleatorio** para desagregar visualmente las burbujas sin alterar los datos subyacentes. Además, ante la propuesta del agente de incluir un gráfico de barras apiladas genérico, decidimos reemplazarlo por una visualización inspirada en el **Cuarteto de Anscombe**, contrastando las pendientes de amortización de cada estrato tarifario.

Hito 6: Pipeline ETL Idempotente y Transaccional (Actividad 4)

- **Interacción Inicial:**
  Prompt enviado: "Necesito estructurar `pipeline/pipeline_tp2.py` como un script autónomo. Revisá las condiciones de desaprobación de la Sección 7 del enunciado."
  Propuesta del Agente: El agente propuso una función simple con `df.to_sql(..., if_exists='replace')`.
- **Intervención y Corrección:**
  Identificación de riesgos de producción: Señalamos dos fallas críticas en la propuesta:
  1. `if_exists='replace'` destruye la tabla DDL original, eliminando tipos de datos específicos de SQL Server (`DECIMAL(5,2)`, `DATETIME2`), claves primarias y restricciones de integridad.
  2. Si la carga falla en medio del proceso, la tabla queda en un estado corrupto o vacía.
- **Solución de Diseño Implementada:**
  Exigimos implementar una estrategia transaccional atómica *Truncate-and-Load* mediante `with engine.begin() as connection:`, ejecutando un `DELETE` controlado previo al `to_sql(..., if_exists='append')`. Si ocurre cualquier error, la transacción revierte automáticamente (`ROLLBACK`). Demostramos la idempotencia ejecutando el script dos veces consecutivas y verificando exactamente 1.000 filas finales en la base.

---

3. Crítica de Código: Implementación Inicial Propuesta vs. Versión Propia Optimizada

A continuación se compara una porción crítica del pipeline de ingeniería de datos: la función de transformación, cálculo de ratios de recupero y control de calidad.

Versión A — Propuesta Inicial del Asistente (Imperativa, Vulnerable e Ineficiente)

```
# Propuesta inicial generada por la IA
def procesar_recupero_ia(df):
    # La IA no valida precondiciones de calidad ni tipos
    ratios = []
    breakevens = []

    # Bucle fila por fila (anti-patrón en Pandas)
    for index, row in df.iterrows():
        # Cálculo de costo con riesgo si replacement_cost es nulo
```

```
        costo_total = row['inventory_count'] * row['replacement_cost']

        # Manejo procedural de división por cero
        if costo_total > 0:
            ratio = row['total_revenue'] / costo_total
        else:
            ratio = 0.0

        ratios.append(round(ratio, 3))

        # Cálculo de break-even sin contemplar rental_rate <= 0
        be = row['replacement_cost'] / row['rental_rate']
        breakevens.append(int(be) + 1)

    df['ratio_recupero'] = ratios
    df['breakeven_alquileres'] = breakevens
    return df
```

Versión B — Implementación Definitiva del Equipo (Vectorizada, Defensiva y Tipada)

```
# Versión definitiva implementada en pipeline/pipeline_tp2.py
def transformar(dataframe: pd.DataFrame, ruta_clasificacion: Path = DATA_PATH) ->
pd.DataFrame:
    """Aplica validaciones de calidad, cálculo vectorizado y enriquecimiento
multi-fuente."""
    # 1. Programación defensiva: validación previa estricta (falla temprano si hay
corrupción)
    validar_calidad(dataframe)

    resultado = dataframe.copy()
    sin_stock = resultado["inventory_count"].eq(0)

    # 2. Enriquecimiento multi-origen validado (Segundo origen: archivo plano CSV)
    clasificacion_df = pd.read_csv(ruta_clasificacion)
    resultado = resultado.merge(
        clasificacion_df,
        on="rental_rate",
        how="left",
        validate="many_to_one",
        indicator=True,
    )
    if not (resultado["_merge"] == "both").all():
        raise ValueError("Existen tarifas de alquiler sin clasificación comercial
asignada.")
    resultado.drop(columns=["_merge"], inplace=True)

    # 3. Métricas derivadas 100% vectorizadas con primitivas de NumPy en C (sin
bucles)
    resultado["costo_inventario_total"] = (
        resultado["inventory_count"] * resultado["replacement_cost"]
    ).round(2)
```

```
    resultado["ratio_recupero"] = np.where(
        resultado["costo_inventario_total"] > 0,
        (resultado["total_revenue"] /
resultado["costo_inventario_total"]).round(3),
        0.0,
    )

    resultado["breakeven_alquileres"] = np.where(
        resultado["rental_rate"] > 0,
        np.ceil(resultado["replacement_cost"] /
resultado["rental_rate"]).astype(int),
        0,
    )

    # 4. Clasificación semántica de negocio
    resultado["estado_recupero"] = np.select(
        [sin_stock, resultado["ratio_recupero"] >= 1.0],
        ["Sin inventario", "Recuperó inversión (≥ 1.0x)"],
        default="No recuperó (< 1.0x)",
    )

    return resultado
```

Cuadro Comparativo de Arquitectura de Código

Criterio de Evaluación	Versión Propuesta por IA (Versión A)	Versión Definitiva del Equipo (Versión B)	Justificación Técnica de la Mejora
Eficiencia Algorítmica	Bucle procedural con `iterrows()`.	100% Vectorizado (`np.where`, `np.select`, `np.ceil`).	La cátedra prohíbe bucles explícitos. `iterrows()` genera una sobrecarga masiva de instanciación de objetos `Series`. Nuestra versión corre en microsegundos aprovechando primitivas vectorizadas en C.
Programación Defensiva	Ninguna aserción previa. Confía ciegamente en los datos de entrada.	Invocación a `validar_calidad()` con aserciones sobre claves primarias, nulos y tipos.	Aplica el principio de diseño *Fail-Fast*: ante cualquier corrupción en la extracción, aborta la ejecución antes de tocar la base de datos o generar cálculos erróneos.
Manejo de Casos de Borde	Bloque `if/else` rudimentario. Riesgo de	Máscara booleana (`sin_stock`) y tratamiento explícito con `np.where`.	Protege matemáticamente las operaciones en las 42

Criterio de Evaluación	Versión Propuesta por IA (Versión A)	Versión Definitiva del Equipo (Versión B)	Justificación Técnica de la Mejora
	`ZeroDivisionError` si `rental_rate` fuera 0.		películas sin copias y evita excepciones de punto flotante en tiempo de ejecución.
Integración Multi-Fuente	Ninguna. Opera sobre un dataset aislado.	`pd.merge()` con archivo plano complementario y validación `validate='many_to_one'`.	Cumple con el requerimiento de la Actividad 4 de orquestar al menos dos fuentes heterogéneas, validando que ninguna fila quede sin mapear.
Semántica del Dominio	Devuelve números crudos sin contexto interpretativo.	Incorpora la columna categórica `estado_recupero` (`Sin inventario`, `Recuperó`, `No recuperó`).	Transforma un cálculo puramente contable en información categorizada accionable para el análisis de negocio.

4. Conclusiones Analíticas Erróneas Sugeridas por la IA y Detección Crítica

Uno de los aportes más relevantes de nuestro rol como supervisores consistió en identificar y refutar conclusiones analíticas erróneas sugeridas por el modelo, las cuales sonaban formalmente sofisticadas pero carecían de validez ante los datos reales y la teoría estadística de la Unidad 5.

Caso 1: La Falacia de la Elasticidad Continua y Recomendación de Precios Intermedios

- **Conclusión Propuesta por la IA:**
  Tras calcular una correlación lineal débil entre precio de alquiler y cantidad de alquileres ($r = -0.053$), el asistente redactó:

  > *"Dado que el coeficiente de Pearson presenta signo negativo ($r = -0.053$), se evidencia una ligera pendiente descendente en la curva de demanda. El videoclub debería optimizar sus ingresos mediante una estrategia de precios dinámicos continuos, testeando valores intermedios como $1.99 o $3.49 para capturar excedente del consumidor."*

- **Cómo detectamos el error y criterio aplicado:**
  Al cruzar esta afirmación con el perfilado de datos de la Actividad 2, observamos que la variable `rental_rate` en Sakila **solo asume 3 valores discretos fijos**: **$0.99 (341 películas)**, **$2.99 (323 películas)** y **$4.99 (336 películas)**.
  1. *Falta de significancia:* El p-valor asociado a la correlación es $p = 0.100 > 0.05$, lo que indica que estadísticamente no existe correlación lineal entre tarifa y volumen de alquiler.
  2. *Imposibilidad teórica de elasticidad continua:* Como enseña la Unidad 5, no es matemáticamente válido derivar una curva de demanda continua ni calcular una elasticidad-precio puntual ($\epsilon = \frac{\%\Delta Q}{\%\Delta P}$) cuando el precio no es una variable continua sino un atributo discreto de 3 niveles.

3. *Corrección en el informe:* Rechazamos la propuesta de la IA y estructuramos la conclusión como una comparación estricta entre tres segmentos cualitativos (*Económico*, *Estándar*, *Premium*), demostrando inelasticidad de demanda dentro del rango analizado.

Caso 2: Supuesta Causalidad Directa y Sesgo de Variable Omitida

- **Conclusión Propuesta por la IA:**
  El asistente sugirió afirmar en el resumen ejecutivo del informe:

  > *"El precio fijado a una película determina de manera directa su rentabilidad y tasa de recupero, por lo que aumentar unilateralmente el precio de las películas de $0.99 a $4.99 triplicará la velocidad de amortización del catálogo."*

- **Cómo detectamos el error y criterio aplicado:**
  Advertimos que la herramienta incurría en el clásico error de confundir correlación con causalidad y omitir variables determinantes del negocio cinematográfico:
  1. *Variable Omitida:* Las películas asignadas a $4.99 suelen ser producciones con mayor presupuesto, actores de renombre o géneros más taquilleros. El precio de $4.99 actúa como una **señal de categoría**, no como la causa aislada que atrae al público.
  2. *Impacto en el recupero:* Si bien una película a $4.99 genera más recaudación por transacción, también posee copias con costos de reposición mayores. La amortización depende de la relación entre rotación y costo de reposición (`replacement_cost`), no del precio aislado.
  3. *Corrección en el informe:* Incorporamos en el informe final y en la preparación para la defensa oral la advertencia explícita sobre el **sesgo de variable omitida** (*Omitted Variable Bias*).

Caso 3: Propuesta de Eliminar Casos de Borde (Outliers Superiores y Títulos con Ingreso Cero)

- **Conclusión Propuesta por la IA:**
  Durante la fase de limpieza de datos, el modelo sugirió aplicar una regla de depuración automática:

  > *"Para normalizar la distribución y mejorar el coeficiente de determinación (R^2), se recomienda descartar las 42 películas con `total_revenue = 0` por constituir registros vacíos, y truncar los 11 outliers superiores detectados por Tukey ($> \$181.80$) por considerarlos ruido estadístico."*

- **Cómo detectamos el error y criterio aplicado:**
  Reconocimos de inmediato que seguir la sugerencia de la IA destruiría la integridad del catálogo:
  1. Las 42 películas sin recaudación son títulos reales del catálogo que no tenían copias físicas adquiridas en la tienda (`inventory_count = 0`). Excluirlas ocultaría el costo de oportunidad de títulos desaprovechados.
  2. Los 11 outliers superiores son títulos como *TELEGRAPH VOYAGE* ($231.73), que representan el 1% de películas más alquiladas del videoclub. En analítica empresarial, los valores atípicos superiores legítimos no son "ruido": son el núcleo de la facturación del negocio.
  3. *Corrección en el informe:* Mantuvimos la totalidad del dataset (1.000 títulos) y explicamos en el informe la justificación económica de preservar ambos grupos.

---

5. Reflexión Final sobre la Utilización de IA como Agente de Ingeniería

La experiencia de trabajo en este proyecto evidenció un cambio de paradigma sustancial en el uso de herramientas de Inteligencia Artificial para la ingeniería y analítica de datos:

1. **De la Asistencia Pasiva a la Supervisión Agéntica:**
   Al operar dentro de un entorno como Google Antigravity IDE, la IA deja de ser un generador de fragmentos descontextualizados y pasa a comportarse como un **asistente de programación en pareja (*pair-programmer*)**. El valor principal radica en la capacidad de orquestar tareas repetitivas, explorar sintaxis de bibliotecas complejas (`pyodbc`, `SQLAlchemy`, `Seaborn`) y estructurar pipelines modulares.
2. **La Imposibilidad de la Automatización Ciega:**
   La herramienta mostró tendencias recurrentes a aplicar recetas genéricas (uso de `INNER JOIN` destructivos, bucles `iterrows()`, descarte ciego de outliers y supuestos económicos continuos). Sin nuestro **análisis crítico constante**, la IA habría producido un pipeline frágil, violaciones a las consignas de portabilidad y conclusiones de negocio totalmente distorsionadas.
3. **Nuestro Rol en la Dirección Técnica del Proyecto:**
   El éxito del trabajo radicó en formular preguntas precisas, exigir planes previos de implementación, someter las respuestas a repreguntas basadas en la teoría dictada en clase (Unidad 5) y supervisar minuciosamente cada línea de código antes de incorporarla definitivamente al proyecto.

Equipo 6 — ISNSLBV 2026
Agustin Bugnoni — Lorenzo Zárate